

Serialization and Spark Persist

Data Serialization

- Data serialization is the process of converting a data structure or object state into a format that can be stored and reconstructed later in the same or another computer environment. The opposite of serialization is called deserialization.
- Data Serialization is the process of converting complex data structures or objects into a linear sequence of bytes (serialization stream) that can be easily transmitted over a network, stored in a file, or persisted in a database.

- Deserialization is the reverse process of serialization. It involves reconstructing the original data structure or object from its serialized form (byte stream).
- While serialization and deserialization provide flexibility and interoperability, they also come with overhead in terms of processing time and bandwidth, especially for large data sets.

Role of data serialization in Big Data Analytics?

- In big data analytics, data serialization plays a critical role in how data is stored, transmitted, and processed. The size and complexity of big data often requires the use of specialized data serialization formats and techniques to ensure efficient storage and processing.

AVRO and file based data structures

- Apache Avro is a data serialization system that provides a compact binary format for data and a data structure specification language to describe the schema of the data. Avro was developed specifically for big data analytics and is well suited for use in Hadoop and other distributed systems.
- Avro provides several benefits over other data serialization systems, such as XML and JSON. One of the key benefits is that Avro supports a compact binary format, which allows for more efficient storage and faster data transfer compared to text-based formats like XML and JSON.

The Interfaces in distributed file system

- A distributed file system typically provides a number of interfaces that allow users and applications to access and manage data stored in the system. The most common interfaces in distributed file systems are:
- File System API: A set of application programming interfaces (APIs) that allow users and applications to interact with the file system, such as reading and writing files, creating and deleting directories, and managing metadata.
- Network File System (NFS) Protocol: A widely used protocol that allows users to access files stored in a remote file system as if they were stored locally. NFS provides a common set of file operations and is supported by many operating systems and applications.
- Server Message Block (SMB) Protocol: A protocol that allows users to access and manage files on a remote file system using the same operations as they would with a local file system. SMB is commonly used in Windows-based environments.
- Object Storage Interface: A set of APIs that provide a way to store and retrieve unstructured data as objects, rather than as traditional files and directories. Object storage is commonly used in cloud-based storage environments.
- Distributed File System Protocol (DFS): A protocol that provides a unified namespace for a set of distributed file systems, allowing users to access files stored on different nodes as if they were stored in a single file system.

Data Ingest?

- Data ingest is the process of bringing data into a data processing or storage system for further analysis and processing. Data ingest can involve multiple steps, including data acquisition, data extraction, data transformation, and data loading.

Introduction to Spark Storage Level and Cache Persistence

Spark storage level and cache persistence play a crucial role in optimizing the performance of Spark applications.

Understanding the Different Storage Levels in Spark

Disk Storage

Provides fault-tolerance and is suitable for large datasets.

Memory Storage

Enables faster access to data, perfect for frequently accessed datasets.

Off-Heap Storage

Stores data outside the heap, preventing impact on the garbage collection process.

Spark Cache vs Persist

- Using `cache()` and `persist()` methods, Spark provides an optimization mechanism to store the intermediate computation of an RDD, DataFrame, and Dataset so they can be reused in subsequent actions (reusing the RDD, Dataframe, and Dataset computation results).
- Both caching and persisting are used to save the Spark RDD, Dataframe, and Datasets. But, the difference is, RDD `cache()` method default saves it to memory (`MEMORY_ONLY`) whereas `persist()` method is used to store it to the user-defined storage level.
- When you persist a dataset, each node stores its partitioned data in memory and reuses them in other actions on that dataset. And Spark's persisted data on nodes are fault-tolerant meaning if any partition of a Dataset is lost, it will automatically be recomputed using the original transformations that created it.

Advantages of Caching and Persistence

- Cost efficient – Spark computations are very expensive hence reusing the computations are used to save cost.
- Time efficient – Reusing the repeated computations saves lots of time.
- **Execution time** – Saves execution time of the job and we can perform more jobs on the same cluster.

Spark Cache Syntax and Example

- Spark `cache()` method in `Dataset` class internally calls `persist()` method which in turn uses `sparkSession.sharedState.cacheManager.cacheQuery` to cache the result set of `DataFrame` or `Dataset`. Let's look at an example.

```
val columns = Seq("Seqno","Quote")  
  val data = Seq(("1", "Be the change that you wish  
to see in the world"),  
  ("2", "Everyone thinks of changing the world, but  
no one thinks of changing himself."),  
  ("3", "The purpose of our lives is to be happy."))  
  val df = data.toDF(columns:_* )  
  
  val dfCache = df.cache()
```

Spark Persist Syntax and Example

- `persist()`
- `persist(newLevel : org.apache.spark.storage.StorageLevel)`
- Spark persist has two signature first signature doesn't take any argument which by default saves it to MEMORY_AND_DISK storage level and the second signature which takes StorageLevel as an argument to store it to different storage levels.

Unpersist syntax and Example

- We can also unpersist the persistence DataFrame or Dataset to remove from the memory or storage.
- `val dfPersist = dfPersist.unpersist()`

Spark Persist storage levels

All different storage level Spark supports are available at `org.apache.spark.storage.StorageLevel` class. The storage level specifies how and where to persist or cache a Spark DataFrame and Dataset.

MEMORY_ONLY – This is the default behavior of the `RDD cache()` method and stores the RDD or DataFrame as deserialized objects to JVM memory. When there is not enough memory available it will not save DataFrame of some partitions and these will be re-computed as and when required. This takes more memory. but unlike RDD, this would be slower than `MEMORY_AND_DISK` level as it recomputes the unsaved partitions, and recomputing the in-memory columnar representation of the underlying table is expensive

MEMORY_ONLY_SER – This is the same as `MEMORY_ONLY` but the difference being it stores RDD as serialized objects to JVM memory. It takes lesser memory (space-efficient) than `MEMORY_ONLY` as it saves objects as serialized and takes an additional few more CPU cycles in order to deserialize.

MEMORY_ONLY_2 – Same as `MEMORY_ONLY` storage level but replicate each partition to two cluster nodes.

MEMORY_ONLY_SER_2 – Same as `MEMORY_ONLY_SER` storage level but replicate each partition to two cluster nodes.

MEMORY_AND_DISK – This is the default behavior of the DataFrame or Dataset. In this Storage Level, The DataFrame will be stored in JVM memory as a deserialized object. When required storage is greater than available memory, it stores some of the excess partitions into a disk and reads the data from the disk when required. It is slower as there is I/O involved.

MEMORY_AND_DISK_SER – This is the same as `MEMORY_AND_DISK` storage level difference being it serializes the DataFrame objects in memory and on disk when space is not available.

MEMORY_AND_DISK_2 – Same as `MEMORY_AND_DISK` storage level but replicate each partition to two cluster nodes.

MEMORY_AND_DISK_SER_2 – Same as `MEMORY_AND_DISK_SER` storage level but replicate each partition to two cluster nodes.

DISK_ONLY – In this storage level, DataFrame is stored only on disk and the CPU computation time is high as I/O is involved.

DISK_ONLY_2 – Same as `DISK_ONLY` storage level but replicate each partition to two cluster nodes.

Example/Syntax for persist and unpersist methods

```
val dfPersist = df.persist()  
dfPersist.show(false)
```

Using the second signature you can save DataFrame/Dataset to One of the storage levels

MEMORY_ONLY, MEMORY_AND_DISK, MEMORY_ONLY_SER, MEMORY_AND_DISK_SER, DISK_ONLY, MEMORY_ONLY_2, MEMORY_AND_DISK_2

```
val dfPersist = df.persist(StorageLevel.MEMORY_ONLY)  
dfPersist.show(false)
```

This stores DataFrame/Dataset into Memory.

Unpersist syntax and Example

We can also unpersist the persistence DataFrame or Dataset to remove from the memory or storage.

Syntax

```
unpersist() : Dataset.this.type  
unpersist(blocking : scala.Boolean) : Dataset.this.type
```

Example

```
val dfPersist = dfPersist.unpersist()  
dfPersist.show(false)
```

unpersist(Boolean) with boolean as argument blocks until all blocks are deleted.

Exploring the Benefits of Caching Data in Spark

1

Enhanced Performance

Cached data allows for quicker access, significantly improving application speed.

2

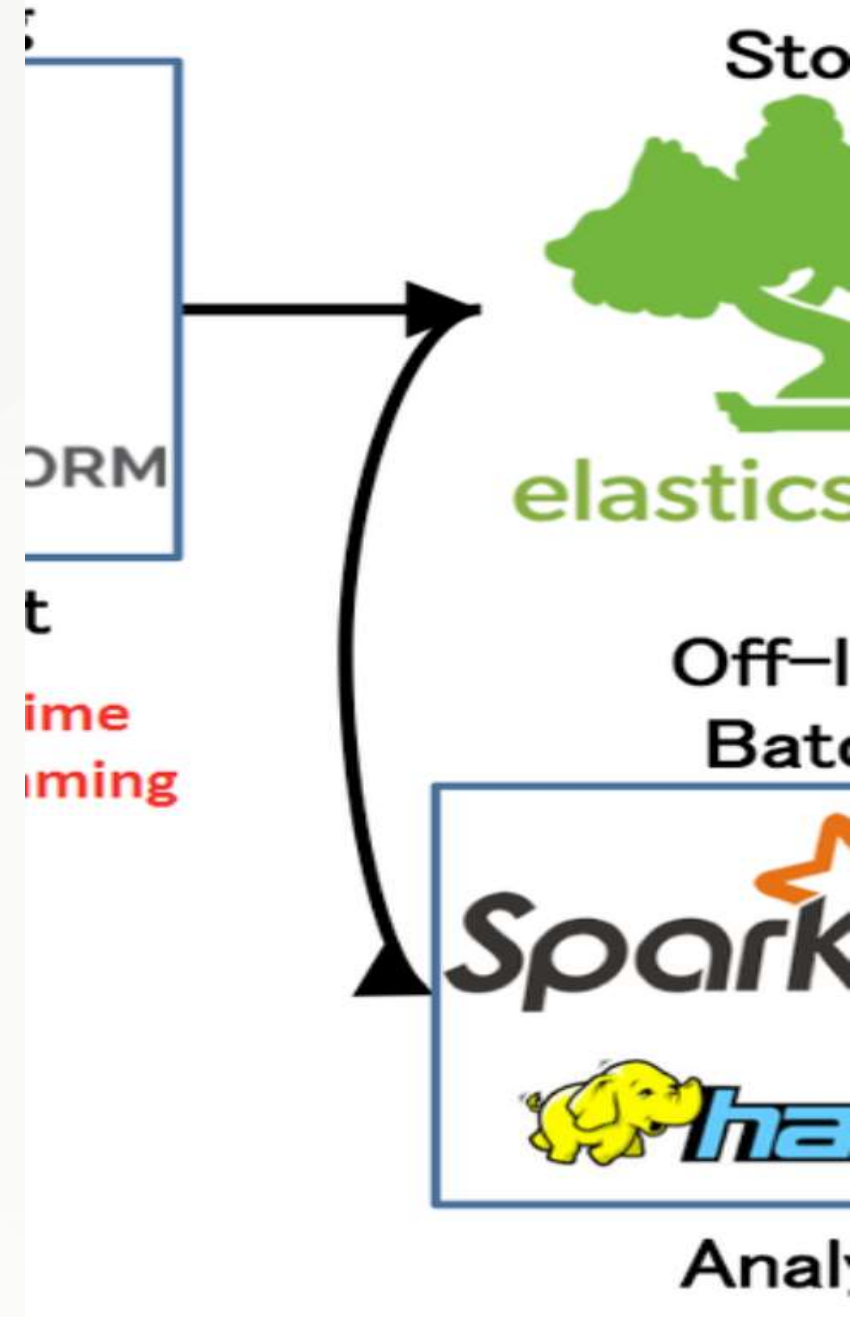
Reduced Recomputation

Caching eliminates the need to recompute data, saving valuable processing resources.

3

Improved User Experience

Users experience reduced latency and faster response times.



How to Cache Data in Spark

Choose Dataset to Cache

Select the specific data to cache based on the application requirements.

Decide Storage Level

Consider the trade-offs between memory, disk, and off-heap storage options.

Execute Caching Command

Use the appropriate method to cache the chosen dataset in Spark.

Eviction Policies for Cache Persistence in Spark

1

LRU (Least Recently Used)

Discards the least recently used data when the memory is full.

2

LFU (Least Frequently Used)

Removes the least frequently accessed data to free up memory space.

3

Custom Policies

Allows developers to define their own eviction strategies based on specific needs.

Monitoring and Managing Cache Persistence in Spark

Spark UI

Provides detailed insights into the caching behavior and storage levels used.

Metrics Tracking

Track cache hit rates, storage usage, and data eviction metrics for optimization.

Best Practices for Using Cache Persistence in Spark

1

Right Data Selection

Choose datasets judiciously for caching to avoid unnecessary memory utilization.

2

Periodic Cache Clearing

Implement strategies to refresh and clear cache to prevent staleness of data.

3

Eviction Monitoring

Regularly monitor and adjust eviction policies to optimize memory usage.



Conclusion and Key Takeaways

Understanding Spark storage levels and cache persistence is essential for maximizing the performance and efficiency of data processing applications. By carefully managing and utilizing caching, developers can significantly enhance the overall execution of Spark jobs.